MGL 7230: Tests Logiciels

Chapitre #3

Test de flux de données

Slides are adapted from Chapter 7 of "Introduction to Software Testing" (2nd Edition) by Ammann and Offutt.

Plan

- Définition et utilisation
- Critères de tests de flux de données
  - All-Def
  - All-Uses
  - All-du-paths
- Flux de données au niveau de la conception
  - Graphes d'appel
  - Paires inter-procédurales def-use
  - Critère Last-def first-use
- Flux de données au niveau de la conception

Critères de flux de données

Objectif : Essayer de garantir que les valeurs sont calculées et utilisées correctement

- Définition (def) : Emplacement où la valeur d'une variable est stockée en mémoire
- Utilisation (use) : Emplacement où l'on accède à la valeur d'une variable

Définitions et utilisations

- **def : un emplacement où une valeur est stockée en mémoire**
  - x apparaît sur le côté gauche d'une affectation (x = 44 ;)
  - x est un paramètre réel dans un site d'appel et la méthode change sa valeur
  - x est un paramètre formel d'une méthode (définition implicite au démarrage de la méthode)
  - x est une entrée d'un programme
- **utilisation :** un emplacement où l'on accède à la valeur de la variable
  - x apparaît sur le côté droit d'une affectation (y = x ;)
  - x apparaît dans un test conditionnel (si (x < 0))
  - x est un paramètre réel d'une méthode
  - x est une sortie du programme
  - x est une sortie d'une méthode dans une instruction de retour

Tableaux: Définitions (def) et Utilisations (use)

- **Les tableaux sont délicats. Considérez la déclaration suivante et deux instructions en C :**

```
int A[10];
A[i]=x+y;
```

- La première instruction définit la variable A.
- La deuxième instruction définit A et utilise i, x et y.
- Alternative : la deuxième instruction définit A[i] et non le tableau A entier.

C-use

- Les utilisations d'une variable qui se produisent dans une expression dans le cadre d'une instruction d'affectation, dans une instruction de sortie, en tant que paramètre dans un appel de fonction et dans des expressions d'indice sont classées comme c-use, où le « c » dans c-use signifie computationnel.

Combien de c-use de x pouvez-vous trouver dans les affirmations suivantes ?

```
z=x+1;
A[x-1]= B[2];
foo(x*x)
output(x);
```

Réponse : 5

p-use

- L'occurrence d'une variable dans une expression utilisée comme condition dans une instruction de branchement telle qu'un if et un while, est considérée comme une p-use. Le « p » dans p-use signifie prédicat.
- Combien de p-use de z et x pouvez-vous trouver dans les affirmations suivantes ?

```
if(z>0){output (x)};
while(z>x){...};
```

Réponse : 3 (2 de z et 1 de x)

p-use : confusion possible

Considérez la déclaration :

```
if(A[x+1]>0){output (x)};
```

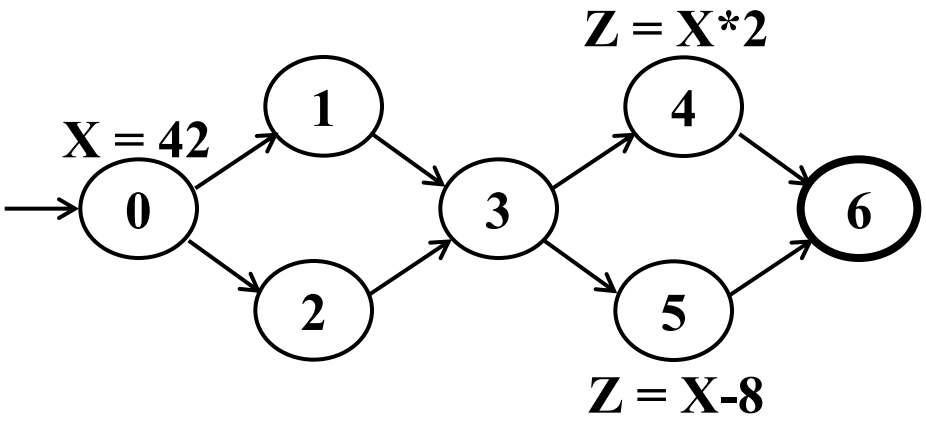
L'utilisation de A est clairement un **p-use**.

L'utilisation de x dans l'indice est-elle une c-use ou p-use ? Discutez-en.

- La source de confusion **est la position de « x » dans l'instruction if**. « x » est utilisé dans une expression qui calcule la valeur d'index de A ; il n'apparaît pas directement dans l'instruction if comme dans if (x>0){…}. On peut donc argumenter dans les deux sens :
- Cette occurrence de x est une **c-use** car elle n'affecte pas directement la condition
- Cette occurrence de x est une **p-use** car elle se produit dans le contexte d'une décision.

8

Critères de flux de données



Defs: def (0) = {X}

def (4) = {Z}

def (5) = {Z}

Uses: use (4) = {X}

use (5) = {X}

9

Paires DU et chemins DU

- def (n) : L'ensemble des variables définies par le nœud n
- use (n) ou use (e) : L'ensemble des variables utilisées par le nœud n ou l'arête e
- du pair: (Def-Use) Une paire d'emplacements (l_i, l_j) telle qu'une variable v soit définie à l_i et utilisée à l_j
- def-clear: Un chemin de l_i to l_j est def-clear par rapport à la variable v si aucune autre valeur n'est donnée à v sur aucun des nœuds ou des arêtes du chemin
- Reach: S'il existe un chemin def-clear défini depuis l_i to l_j par rapport à v, la def de v à l_i **atteint** le use à l_j

Paires DU et chemins DU

- du-path : Un chemin **simple** qui est def-clear par rapport à v depuis une définition de v vers une utilisation de v

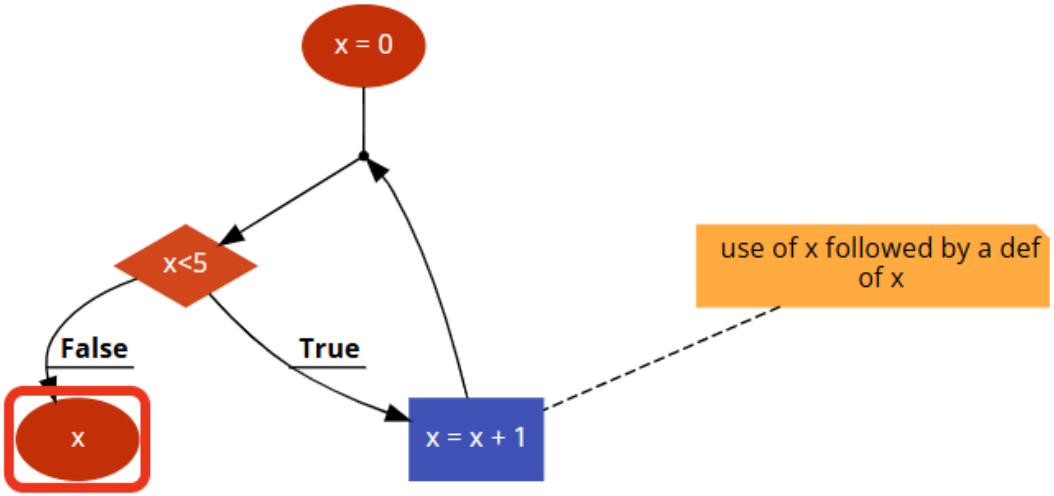
- Ensemble Def-path: du (n, v) – l'ensemble des chemins définis qui commencent à n **(tous les chemins simples définis de n à toutes les utilisations de v)**

- Ensemble Def-pair set: du (n_i, n_j, v) – l'ensemble des chemins **du** de n_i to n_j (**tous les chemins simples def-clear depuis la définition de v à ni vers une utilisation de v à nj**)

Paires DU dans les boucles

- Si une **def** et une **use** apparaissent sur le même nœud, il s'agit alors d'une paire DU uniquement si la définition se produit après l'utilisation et que le nœud est dans une boucle.

```
x = 0;
while (x<5) {
x = x + 1; // use of x followed
  by a def of x
}
return x;
```



Generated using: https://app.code2flow.com/

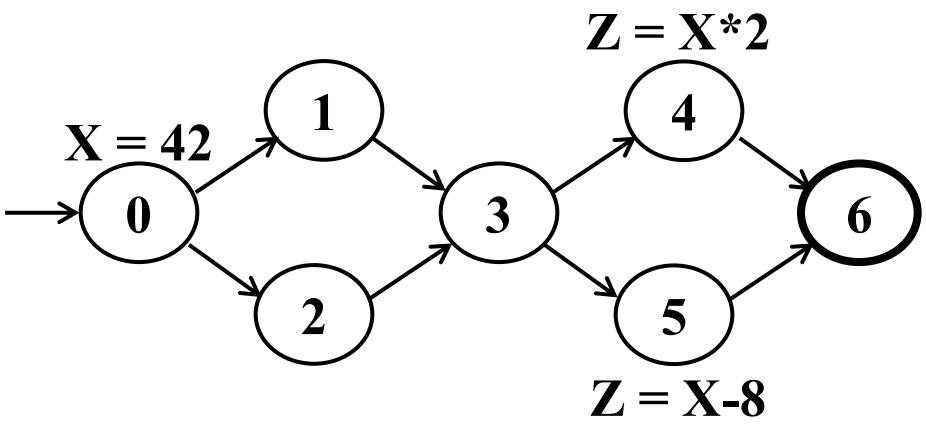
Critères de test du flux de données

1) Tout d'abord, nous nous assurons que chaque **def** atteint au moins une **use.**

<u>All-defs coverage (ADC)</u> :

Pour chaque ensemble de chemins de définition (def-path), S = du(n, v), TR contient au moins un chemin d dans S.

- Nous oblige à parcourir au moins un chemin pour au moins une utilisation



All-defs pour X
[0, 1, 3, 4]

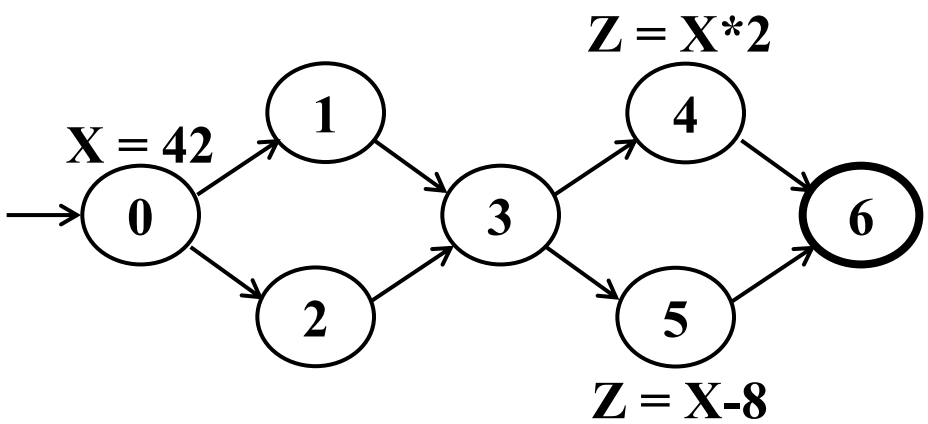
Critères de test du flux de données

2) Ensuite, nous nous assurons que chaque définition atteint toutes les utilisations possibles

Couverture tous usages (AUC) :

Pour chaque ensemble de paires (def-pair). S = du(ni, nj, v), TR contient au moins un chemin d dans S.

Nous oblige à parcourir au moins un chemin pour chaque paire **def-use**



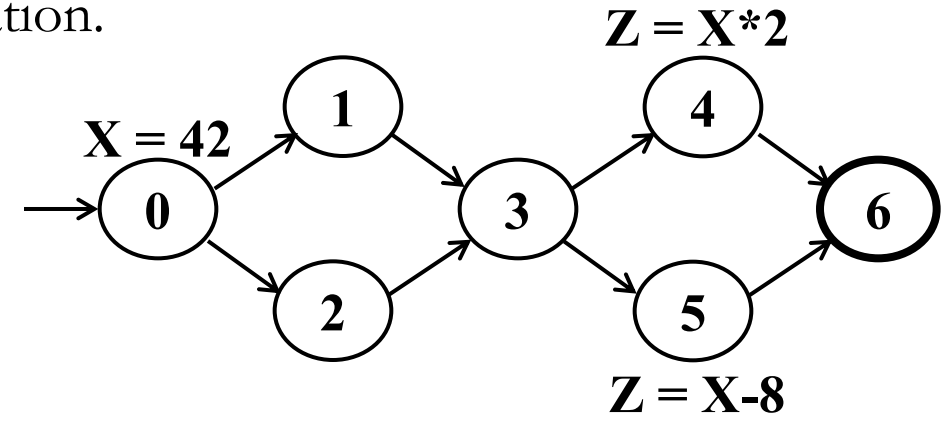
Utilisations multiples pour X
[0, 1, 3, 4]
[0, 1, 3, 5]

Critères de test du flux de données

3) Enfin, nous couvrons tous les chemins entre les définitions et les utilisations (chaque définition atteint toutes les utilisations possibles via tous les chemins du possible)

All-du-paths Coverage (ADUPC): Pour chaque ensemble def-pair S = du (ni, nj, v), TR contient tous les chemins d dans S.

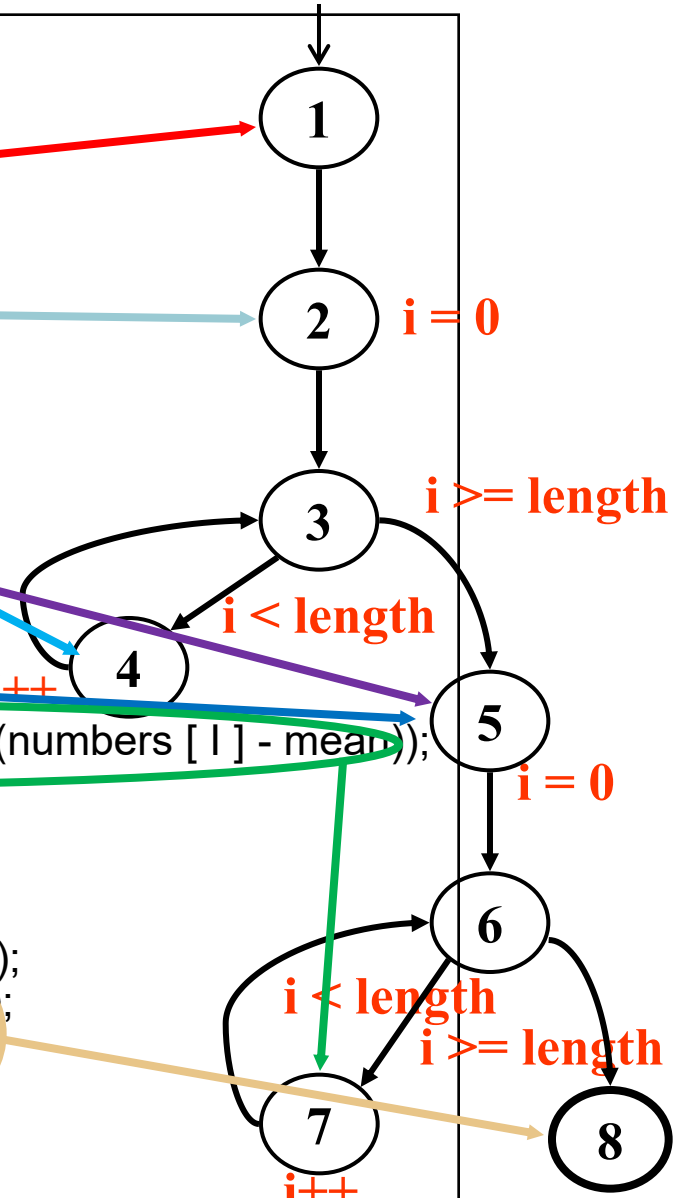
- Cela pourrait être simplement écrit comme « inclure chaque chemin du »
- All-uses nécessite un chemin def-clear simple pour chaque utilisation, alors que All-du-Paths nécessite tous les chemins def-clear pour chaque utilisation.



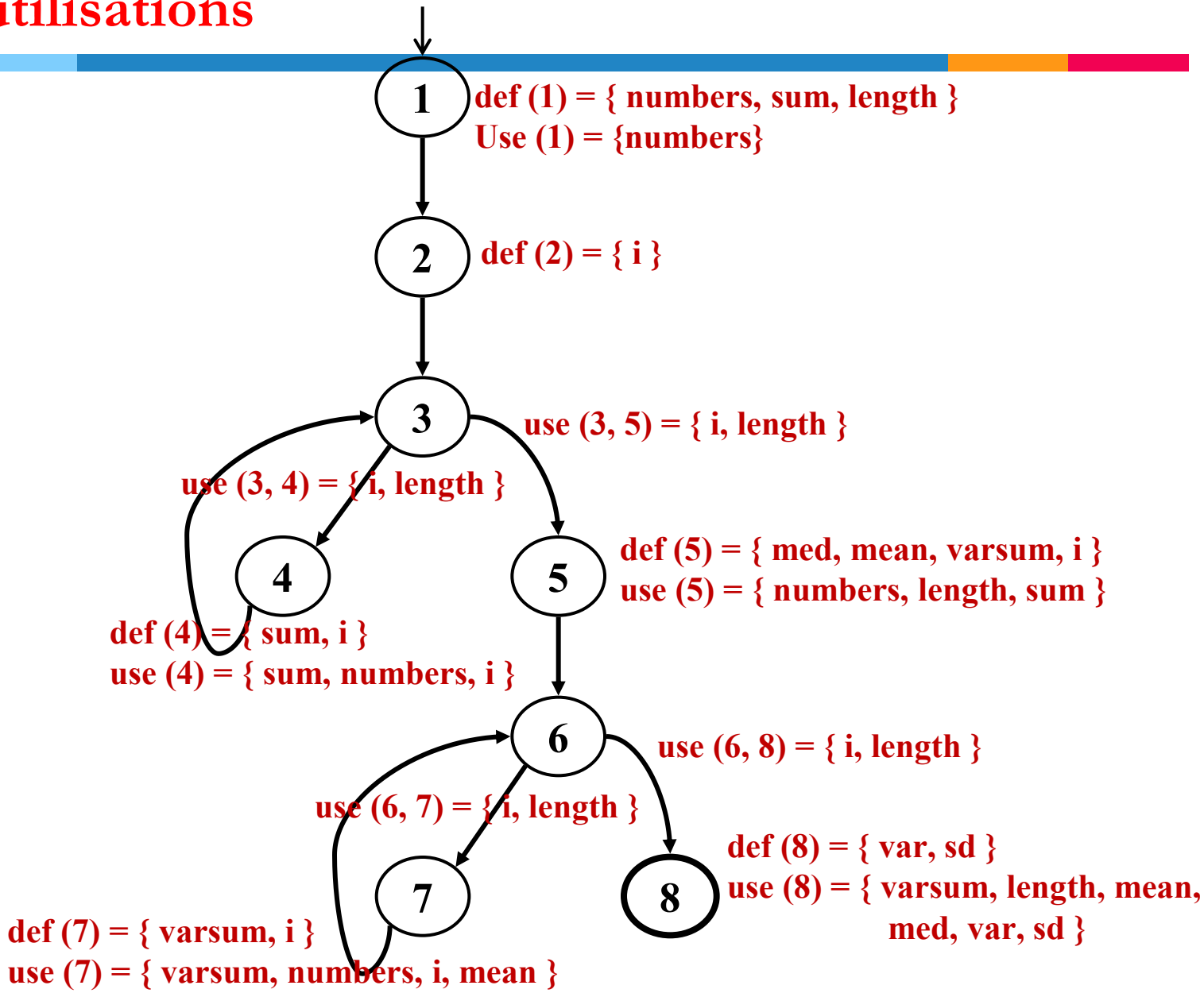
Tous les chemins pour X
[0, 1, 3, 4]
[0, 2, 3, 4]
[0, 1, 3, 5]
[0, 2, 3, 5]

Graphe de flux de contrôle pour les statistiques

```
public static void computeStats (int [ ] numbers)
{
    int length = numbers.length;
    double med, var, sd, mean, sum, varsum;

    sum = 0;
    for (int i = 0; i < length; i++)
    {
        sum += numbers [ i ];
    }
    med   = numbers [ length / 2 ];
    mean = sum / (double) length;

    varsum = 0;
    for (int i = 0; i < length; i++)
    {
        varsum = varsum  + ((numbers [ I ] - mean) * (numbers [ I ] - mean));
    }
    var = varsum / ( length - 1.0 );
    sd  = Math.sqrt ( var );

    System.out.println ("length:                    " + length);
    System.out.println ("mean:                      " + mean);
    System.out.println ("median:                    " + med);
    System.out.println ("variance:                  " + var);
    System.out.println ("standard deviation: " + sd);
}
```



CFG pour les statistiques – avec définitions et utilisations



Tableaux de définitions et d'utilisations pour les statistiques

Node	Def	Use
1	{ numbers, sum, length }	{numbers}
2	{ i }	
3		
4	{ sum, i }	{ numbers, i, sum }
5	{ med, mean, varsum, i }	{ numbers, length, sum }
6		
7	{ varsum, i }	{ varsum, numbers, i, mean }
8	{ var, sd }	{ varsum, length, var, mean, med, var, sd }

Edge	Use
(1, 2)	
(2, 3)	
(3, 4)	{ i, length }
(4, 3)	
(3, 5)	{ i, length }
(5, 6)	
(6, 7)	{ i, length }
(7, 6)	
(6, 8)	{ i, length }

DU Pairs for Stats

variable	DU Pairs
numbers	(1, 4) (1, 5) (1, 7)
length	(1, 5) (1, 8) (1, (3,4)) (1, (3,5)) (1, (6,7)) (1, (6,8))
med	(5, 8)
var	(8, 8)
sd	(8, 8)
mean	(5, 7) (5, 8)
sum	(1, 4) (1, 5) (4, 4) (4, 5)
varsum	(5, 7) (5, 8) (7, 7) (7, 8)
i	(2, 4) (2, (3,4)) (2, (3,5)) ~~(2, 7) (2, (6,7)) (2, (6,8))~~
	(4, 4) (4, (3,4)) (4, (3,5)) ~~(4, 7) (4, (6,7)) (4, (6,8))~~
	(5, 7) (5, (6,7)) (5, (6,8))
	(7, 7) (7, (6,7)) (7, (6,8))

No paths between 8 and 8

defs after use in loop, these are valid DU pairs

No def-clear path … different scope for i

No path through graph from nodes 5 and 7 to 4 or 3

Chemins DU pour les statistiques

variable	DU Pairs	DU Paths
numbers	(1, 4) (1, 5) (1, 7)	[1, 2, 3, 4] [1, 2, 3, 5] [1, 2, 3, 5, 6, 7]
length	(1, 5) (1, 8) (1, (3,4)) (1, (3,5)) (1, (6,7)) (1, (6,8))	[1, 2, 3, 5] [1, 2, 3, 5, 6, 8] [1, 2, 3, 4] [1, 2, 3, 5] [1, 2, 3, 5, 6, 7] [1, 2, 3, 5, 6, 8]
med	(5, 8)	[5, 6, 8]
var	(8, 8)	*No path needed*
sd	(8, 8)	*No path needed*
sum	(1, 4) (1, 5) (4, 4) (4, 5)	[1, 2, 3, 4] [1, 2, 3, 5] [4, 3, 4] [4, 3, 5]

variable	DU Pairs	DU Paths
mean	(5, 7) (5, 8)	[5, 6, 7] [5, 6, 8]
varsum	(5, 7) (5, 8) (7, 7) (7, 8)	[5, 6, 7] [5, 6, 8] [7, 6, 7] [7, 6, 8]
i	(2, 4) (2, (3,4)) (2, (3,5)) (4, 4) (4, (3,4)) (4, (3,5)) (5, 7) (5, (6,7)) (5, (6,8)) (7, 7) (7, (6,7)) (7, (6,8))	[2, 3, 4] [2, 3, 4] [2, 3, 5] [4, 3, 4] [4, 3, 4] [4, 3, 5] [5, 6, 7] [5, 6, 7] [5, 6, 8] [7, 6, 7] [7, 6, 7] [7, 6, 8]

Chemins DU pour les statistiques – Pas de doublons

Il existe 38 chemins DU pour les statistiques, mais seulement 12 uniques

[1, 2, 3, 4]	[4, 3, 4]
[1, 2, 3, 5]	[4, 3, 5]
[1, 2, 3, 5, 6, 7]	[5, 6, 7]
[1, 2, 3, 5, 6, 8]	[5, 6, 8]
[2, 3, 4]	[7, 6, 7]
[2, 3, 5]	[7, 6, 8]

4 s'attendre à ce qu'une boucle ne soit pas « entrée »

6 nécessitent au moins une itération d'une boucle

2 nécessitent au moins deux itérations d'une boucle

Cas de test et chemins de test

Cas de test : nombres = (44) ; longueur = 1
Chemin de test : [1, 2, 3, 4, 3, 5, 6, 7, 6, 8]
Chemins DU supplémentaires couverts
[1, 2, 3, 4] [2, 3, 4] [4, 3, 5] [5, 6, 7] [7, 6, 8]
Les cinq étoiles qui nécessitent au moins une itération d'une boucle

Cas de test : nombres = (2, 10, 15) ; longueur = 3
Chemin de test : [1, 2, 3, 4, 3, 4, 3, 4, 3, 5, 6, 7, 6, 7, 6, 7, 6, 8]
Chemins DU parcourus
[4, 3, 4] [7, 6, 7]
Les deux étoiles qui nécessitent au moins deux itérations d'une boucle

D'autres chemins DU nécessitent des tableaux de longueur 0 pour ignorer les boucles
Mais la méthode échoue avec une exception index out of bounds…
med = numbers [length / 2];

Un défaut a été trouvé

Activité

```
1  public class DemoDU {
2    public static void main(String[] args) {
3      int a = 5;
4      int b = 2;
5      int c = 0;
6      if (a > b) {
7        c = a - b;
8      } else {
9        c = b - a;
10     }
11     int i = 0;
12     while (i < 3) {
13       c = c + i;
14       i = i + 1;
15     }
16     System.out.println(c);
17   }
18 }
```

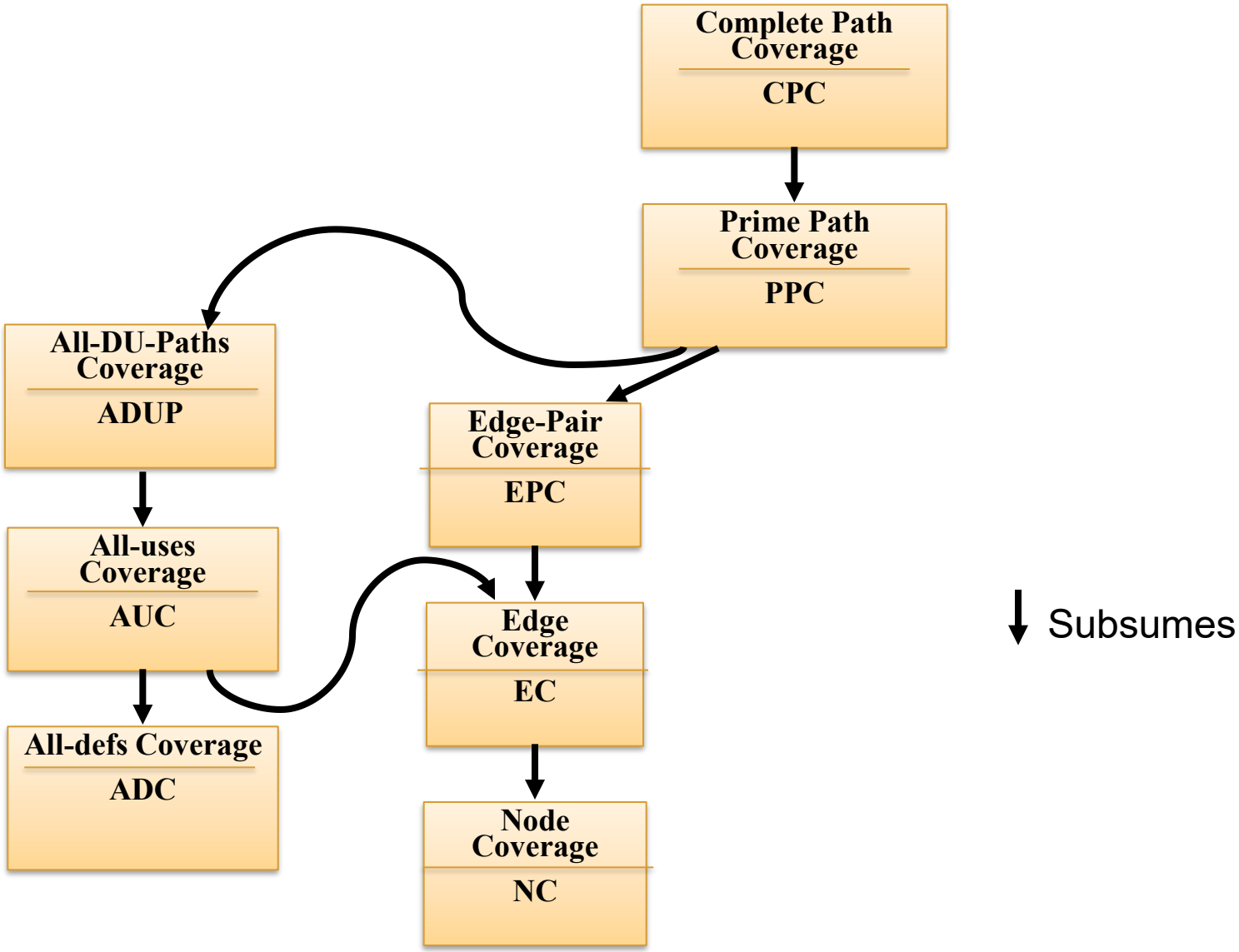
1. Listez toutes les définitions et utilisations des variables (utilisez les numéros de ligne)

2. Calculer tous les chemins **def-use** pour toutes les variables

Comparaison des critères avec la subsomption

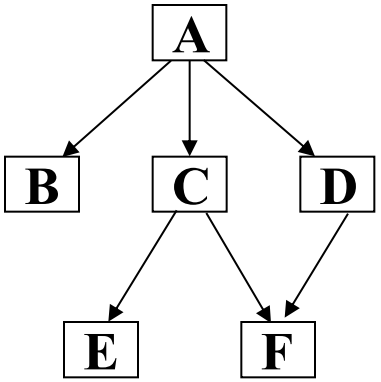
- Critères de sélection : Un critère de test C1 englobe C2 si et seulement si chaque ensemble de cas de test qui satisfait le critère C1 satisfait également C2
- Doit être vrai pour chaque ensemble de cas de test
- *Exemple* : Si un ensemble de tests a couvert toutes les branches d'un programme (a satisfait au critère de branche), alors l'ensemble de tests est assuré d'avoir également couvert toutes les instructions

Critères de couverture graphe Subsomption



Conception structurelle

- Les graphes sont la représentation la plus courante pour les tests de conception structurelle
  - Nœuds : Unités (en Java – méthodes)
  - Arêtes : Appels aux unités

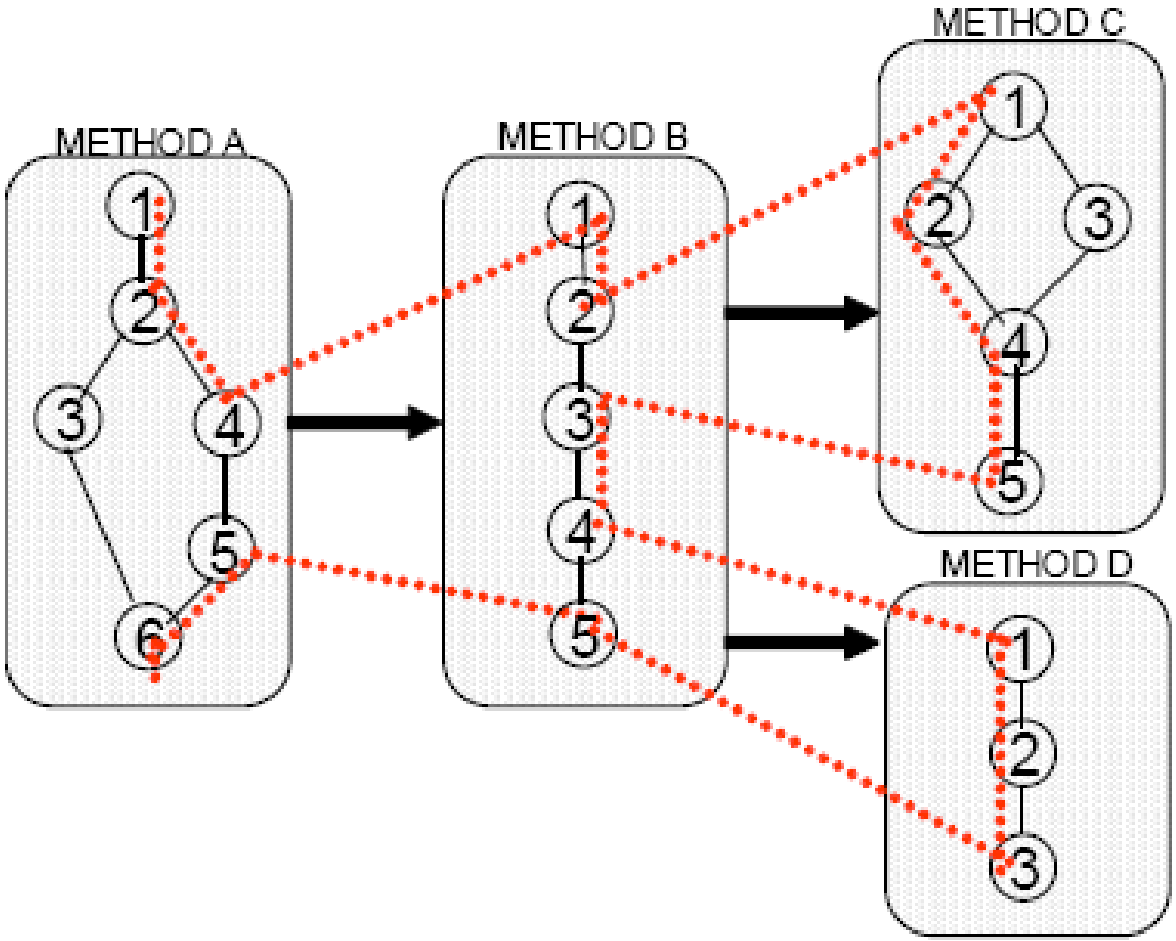


Exemple de graphe d'appel

Couverture des nœuds : appeler chaque unité au moins une fois (couverture des méthodes)

Couverture des arêtes: exécuter chaque appel au moins une fois (couverture d'appel)

Graphe d'appel(Call Graph)



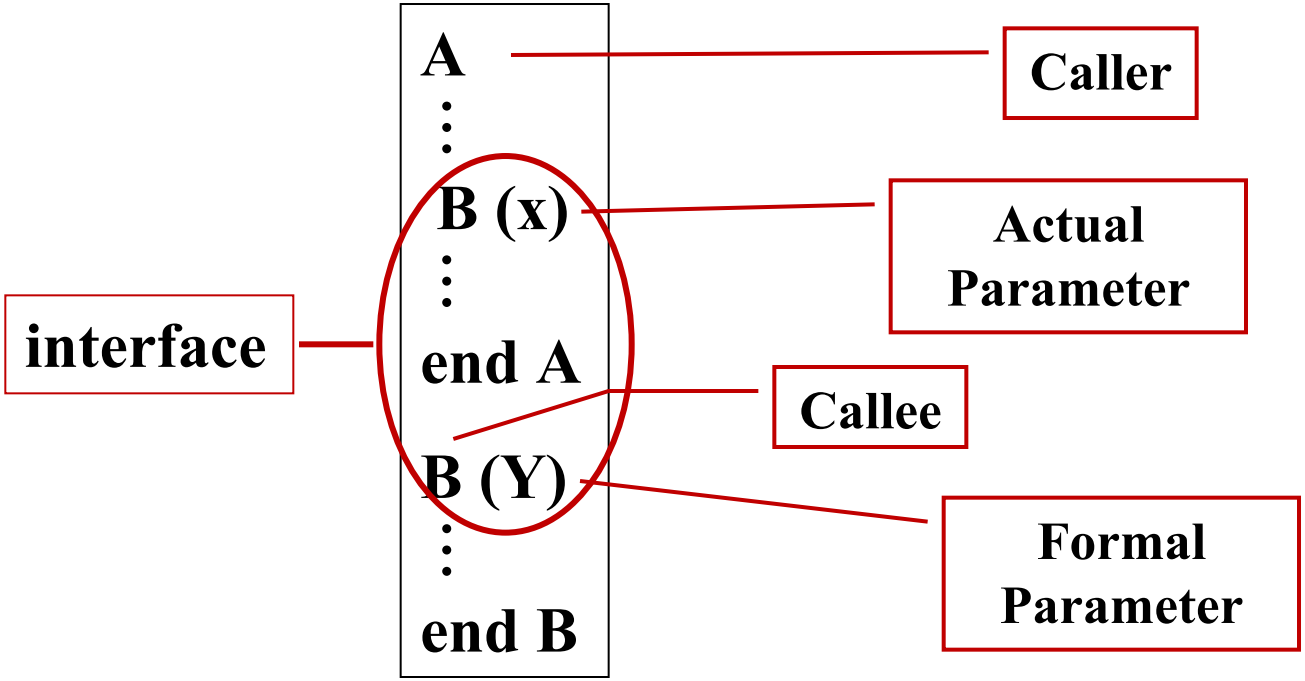
Flux de données au niveau de la conception

- Les couplages de flux de données entre les unités et les classes sont plus compliqués que les couplages de flux de contrôle
  - Lorsque des valeurs sont transmises, elles « changent de nom »
  - De nombreuses façons différentes de partager des données
  - Il peut être difficile de trouver des définitions et des utilisations – il est très difficile de trouver les utilisations auxquelles une définition peut accéder
- **Partage de données :**
  - **Couplage de paramètres :** paramètres transmis lors des appels.
  - **Couplage de données partagées :** deux unités accèdent au même objet de données en tant que variable globale
  - **Couplage de périphériques externes :** deux unités accèdent au même support externe tel qu'un fichier.
- **Variables de couplage** : Variables définies dans une unité et utilisées dans une autre.

Flux de données au niveau de la conception

- Appelant : une unité qui invoque une autre unité
- Appelé : l'unité appelée
- Site d'appel : instruction ou nœud où l'appel apparaît
- Paramètre réel : variable dans l'appelant
- Paramètre formel : variable dans l'appelé

Exemple de site d'appel



Flux de données au niveau de la conception

- Appliquer des critères de flux de données aux paires de définition-utilisation entre les unités est trop coûteux
- Trop de possibilités
- Mais il s'agit de tests d'intégration, et nous ne nous intéressons qu'à l'interface…

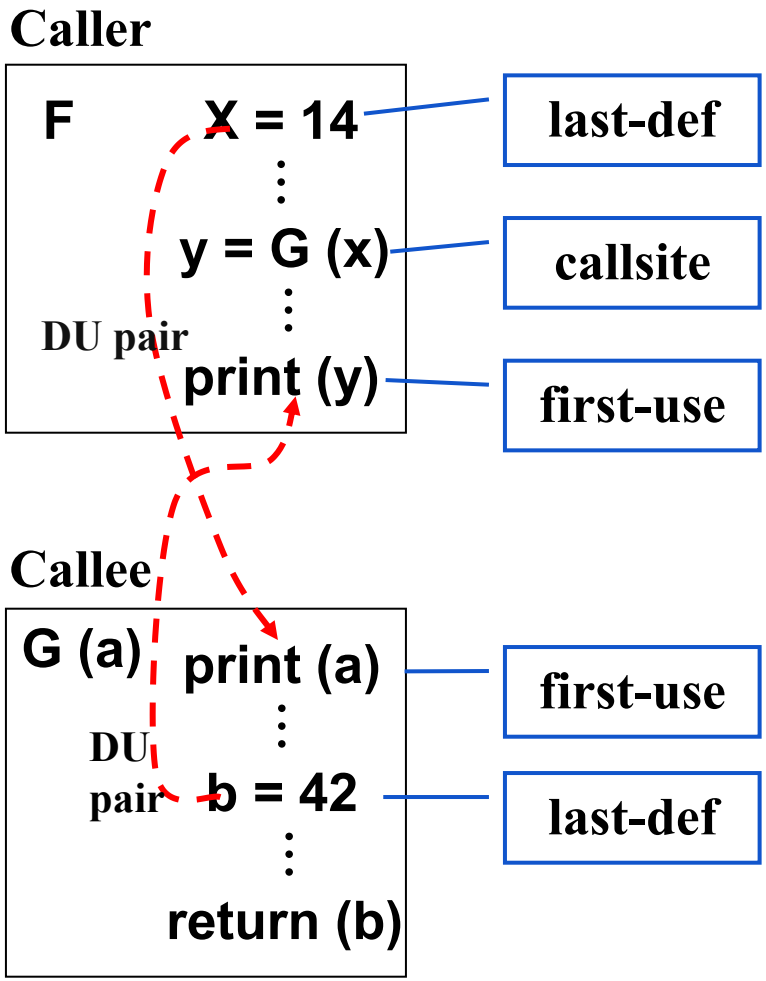
Paires DU inter-procédurales

- Si nous nous concentrons sur l'interface, nous devons simplement considérer les dernières définitions (**last definitions**) des variables avant les appels et les retours et les premières utilisations (**first uses**) à l'intérieur des unités et après les appels

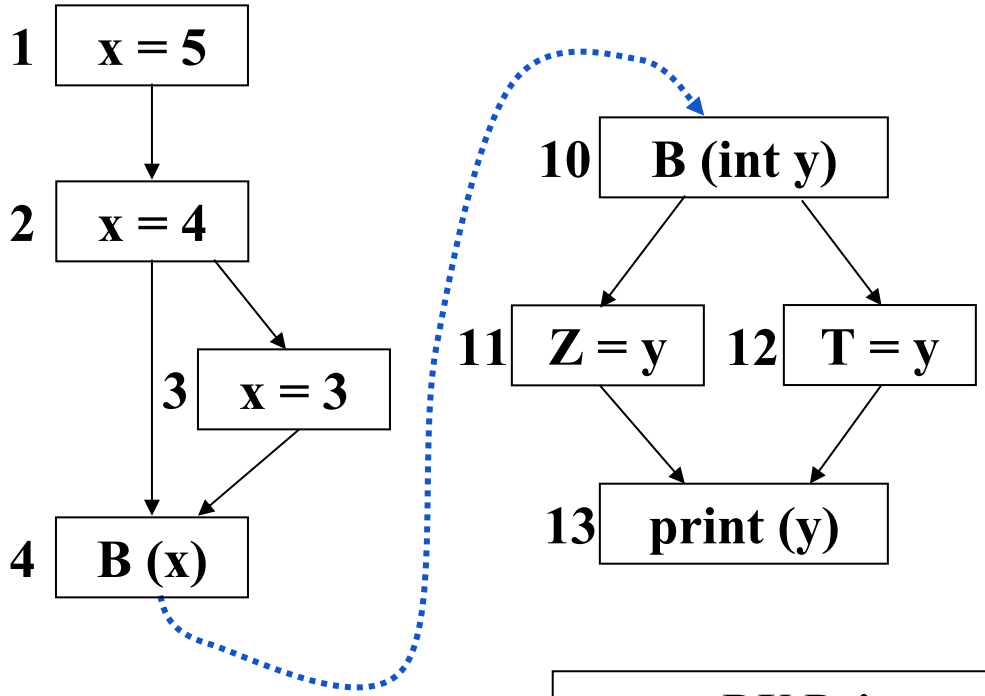
Paires DU inter-procédurales

- Last-def : l'ensemble des nœuds qui définissent une variable x et qui possède un chemin def-clear depuis le nœud via un site d'appel jusqu'à une utilisation dans l'autre unité
  - Peut être de l'appelant vers l'appelé (paramètre ou variable partagée) ou de l'appelé vers l'appelant en tant que valeur de retour
- Use-clear path: : un chemin entre les nœuds i et j est use-clear par rapport à la variable v si pour chaque nœud k sur le chemin entre i et j, v n'est pas dans use(k).
- First-use : l'ensemble des nœuds qui ont des utilisations d'une variable y et pour lesquels il existe un chemin def-clear et un chemin use-clear depuis le site d'appel vers les nœuds.

Exemple de paires DU inter-procédurales



Exemple de paires DU inter-procédurales



Last Defs
2, 3

First Uses
11, 12

DU Pairs

(A, x, 2)—(B, y, 11)

(A, x, 2)—(B, y, 12)

(A, x, 3)—(B, y, 11)

(A, x, 3)—(B, y, 12)

Exemple – Quadratique

```
1 // Program to compute the quadratic root for
two numbers
2 import java.lang.Math;
3
4 class Quadratic
5 {
6  private static float Root1, Root2;
7
8  public static void main (String[] argv)
9  {
10     int X, Y, Z;
11     boolean ok;
12     int controlFlag = Integer.parseInt (argv[0]);
13     if (controlFlag == 1)
14     {
15         X = Integer.parseInt (argv[1]);
16         Y = Integer.parseInt (argv[2]);
17         Z = Integer.parseInt (argv[3]);
18     }
19     else
20     {
21         X = 10;
22         Y = 9;
23         Z = 12;
24     }
```

```
25         ok = Root (X, Y, Z);
26         if (ok)
27           System.out.println
28               (“Quadratic: ” + Root1 + Root2);
29         else
30           System.out.println (“No Solution.”);
31   }
32
33 // Three positive integers, finds quadratic root
34   private static boolean Root (int A, int B, int C)
35   {
36      float D;
37      boolean Result;
38      D = (float) Math.pow ((double)B,
            (double2-4.0)*A*C );
39      if (D < 0.0)
40      {
41         Result = false;
42         return (Result);
43      }
44      Root1 = (float) ((-B + Math.sqrt(D))/(2.0*A));
45      Root2 = (float) ((-B – Math.sqrt(D))/(2.0*A));
46      Result = true;
47      return (Result);
48   } / /End method Root
49
50   } // End class Quadratic
```

Exemple – Quadratique

```
// Program to compute the quadratic root for two numbers
import java.lang.Math;

class Quadratic
{
 private static float Root1, Root2;

 public static void main (String[] argv)
 {
    int X, Y, Z;
    boolean ok;
    int controlFlag = Integer.parseInt (argv[0]);
    if (controlFlag == 1)
    {
       X = Integer.parseInt (argv[1]);
       Y = Integer.parseInt (argv[2]);
       Z = Integer.parseInt (argv[3]);
    }
    else
    {

       X = 10;
       Y = 9;
       Z = 12;
    }
```

shared variables

last-defs

first-use

first-use

last-def

last-defs

```
25          ok = Root (X, Y, Z);
26          if (ok)
27            System.out.println
28                 (“Quadratic: ” + Root1 + Root2);
29          else
30             System.out.println (“No Solution.”);
31    }
32
33    // Three positive integers, finds the quadratic root
34    private static boolean Root (int A, int B, int C)
35    {
36       float D;
37       boolean Result;
38       D = (float) Math.pow ((double)B, (double2-4.0)*A*C);
39       if (D < 0.0)
40       {
41          Result = false;
42          return (Result);
43       }
44       Root1 = (float) ((-B + Math.sqrt(D)) / (2.0*A));
45       Root2 = (float) ((-B – Math.sqrt(D)) / (2.0*A));
46       Result = true;
47       return (Result);
48    } / /End method Root
49
50 } // End class Quadratic
```

Quadratique – Couplage de paires DU

Paires d'emplacements : nom de la méthode, nom de la variable, instruction method name, variable name, statement

(main (), X, 15) – (Root (), A, 38)

(main (), Y, 16) – (Root (), B, 38)

(main (), Z, 17) – (Root (), C, 38)

(main (), X, 21) – (Root (), A, 38)

(main (), Y, 22) – (Root (), B, 38)

(main (), Z, 23) – (Root (), C, 38)

(Root (), Root1, 44) – (main (), Root1, 28)

(Root (), Root2, 45) – (main (), Root2, 28)

(Root (), Result, 41) – (main (), ok, 26)

(Root (), Result, 46) – (main (), ok, 26)

Notes sur le couplage des flux de données

- Initialisations implicites de variables de classe et globales

- **Les paires DU transitives** sont trop coûteuses à gérer
  - A appelle B, B appelle C, et il existe une variable définie dans A et utilisée dans C

- **Tableaux :** une référence à un élément est considérée comme une référence à tous les éléments

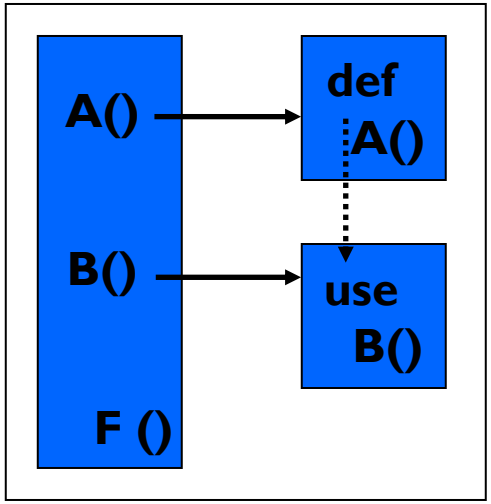
Définitions supplémentaires

- Héritage : Si la classe B hérite de la classe A, alors toutes les variables et méthodes de A sont implicitement dans B, et B peut en ajouter d'autres
  - A est le parent ou l'ancêtre
  - B est l'enfant ou le descendant
- Une référence d'objet *obj* déclarée comme étant de type A peut se voir attribuer un objet de type A, B ou l'un des descendants de B
  - Type déclaré : Type utilisé dans la déclaration : *A obj* ;
  - Type réel : Type utilisé dans l'affectation d'objet : *obj = new B();*
- Variables de classe (état) : Les variables déclarées au niveau de la classe, souvent privées

Héritage, polymorphisme et liaison dynamique

- Les connexions de contrôle et de données supplémentaires rendent l'analyse du flux de données plus complexe
- Les unités de définition et d'utilisation peuvent se trouver dans des hiérarchies d'appel différentes
- Lorsque des hiérarchies d'héritage sont utilisées, une définition dans une unité peut atteindre des utilisations dans n'importe quelle classe de la hiérarchie d'héritage
- Avec la liaison dynamique, le même emplacement peut atteindre des utilisations différentes selon le type actuel de l'objet utilisé
- Le même emplacement peut avoir des définitions ou des utilisations différentes à différents moments de l'exécution !

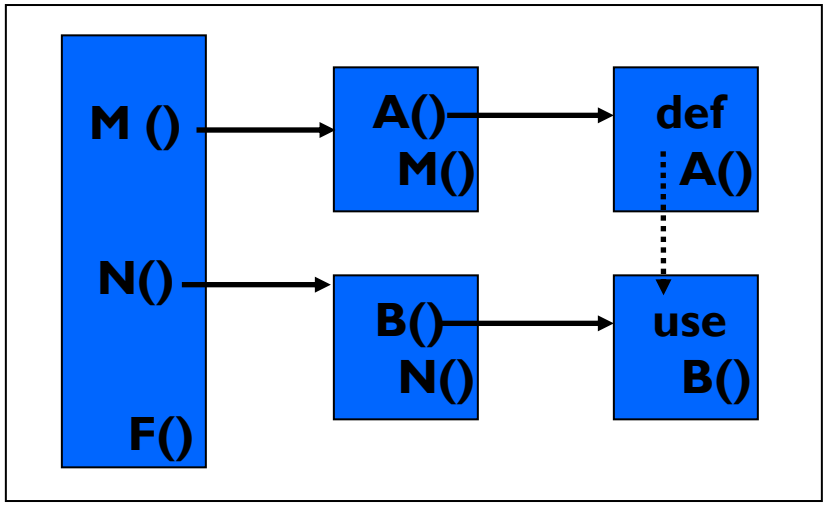
Types de paires Def-Use



object-oriented direct coupling data flow

- Une méthode de couplage, F(), appelle deux méthodes, A() et B().
- A() définit une variable et B() l'utilise.
- Pour que la référence de la variable soit la même, A() et B() doivent être appelés via la même référence d'objet, ex. o.A() et o.B(), sont appelés via le contexte d'instance de o.
- Si les appels ne sont pas effectués dans le même contexte d'instance, la définition et l'utilisation seront faites pour différentes instances de la variable.

Types de paires Def-Use



object-oriented indirect coupling data flow

- La méthode de couplage F() appelle les méthodes M() et N(), qui appellent à leur tour deux autres méthodes, A() et B().
- La definition et l'usage sont dans A() et B(), **donc la référence est indirecte.**
- **Nettement plus complexe que pour les du-paires directes.**
- Il peut y avoir plus d'un appel entre la méthode de couplage et les méthodes avec la definition et l'utilisation.
- En plus, A() et B() peuvent appartenir à la même classe, ou bien être dans des classes différentes et accéder aux mêmes variables globales.

Résumé du flux de données OO

- Les définitions et les utilisations peuvent être dans la même classe ou dans des classes différentes
- Les chercheurs ont appliqué les tests de flux de données à la situation OO à couplage direct
  - N'a pas été utilisé dans la pratique
  - Aucun outil disponible
- Les tests de flux de données à couplage indirect n'ont pas été testés ni dans la recherche ni dans la pratique
  - Le coût de l'analyse peut être prohibitif